

計算物理学II

第2回 コマンド操作とデータの可視化

秋山 進一郎

2026年10月9日

授業日の確認

- ・ 全10回

- ・ 第1回：10月2日（金）

- ・ 第2回：10月9日（金）

- ・ 第3回：10月16日（金）

- ・ 第4回：10月23日（金）

- ・ 第5回：11月6日（金）

- ・ 第6回：11月13日（金）

- ・ 第7回：11月20日（金）

- ・ 第8回：12月4日（金）

- ・ 第9回：12月11日（金）

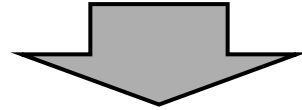
- ・ 第10回：12月18日（金）

今日の授業の目標

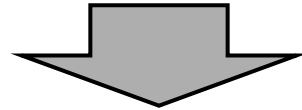
- **引き続き, Terminalの操作に慣れていく**
 - **コマンド操作**
 - **matplotlibによる図の作成**

毎回の授業の流れ

“compphy2”に移動する



“git pull origin main”で演習資料を入手



“lecture2”の中の資料を開いて演習開始

本日の演習内容

- コマンドを使ったファイル操作について
 - 前回の復習といくつかの新しいコマンド (cp, mv, rm, grep, awk)
- matplotlibによる図の作成
 - 関数のプロット
 - 数値データのプロット
 - スクリプトの活用

前回学んだコマンドの復習 1/2

- ・まずは“compphys2”の中にいる状態にしましょう

```
akiyama@~/compphys2$
```

- ・“compphys2”の中に今日の演習用ディレクトリを“work2”という名前で作ります

```
akiyama@~/compphys2$ mkdir work2
```

- ・作った“work2”に移動します

```
akiyama@~/compphys2$ cd work2  
akiyama@~/compphys2/work2$
```

前回学んだコマンドの復習 2/2

- “test.txt”というファイルを作しましょう

```
akiyama@~/compphys2/work2$ touch test.txt
```

- “test.txt”を好きなエディタ（emacs, vi, VS code, …）を使って編集してみよう
 - 「test」と入力して保存し, エディタを閉じましょう
 - lsで編集内容の確認します

```
akiyama@~/compphys2/work2$ cat test.txt  
test
```

ファイルのコピー：cpコマンドを使ってみる

- ・ cpコマンドを使ってファイルのコピーをしてみます
 - ・ “test.txt”を“cp_test.txt”という名前でコピーする
 - ・ コマンドの構造は, “cp コピーしたいファイルのパス コピー先のファイルのパス”
 - ・ パスの説明は後で. まずは以下のコマンドを実行してみましょう

スペース

```
akiyama@~/compphys2/work2$ cp ./test.txt ./cp_test.txt
```

- ・ lsを使って“cp_test.txt”が作成されたことを確認してみましょう
- ・ catを使って“cp_test.txt”の中身を確認し, コピーされていることを確認しましょう

```
akiyama@~/compphys2/work2$ ls
cp_test.txt test.txt
akiyama@~/compphys2/work2$ cat cp_test.txt
test
```

ファイルのコピー：cpコマンドを使ってみる

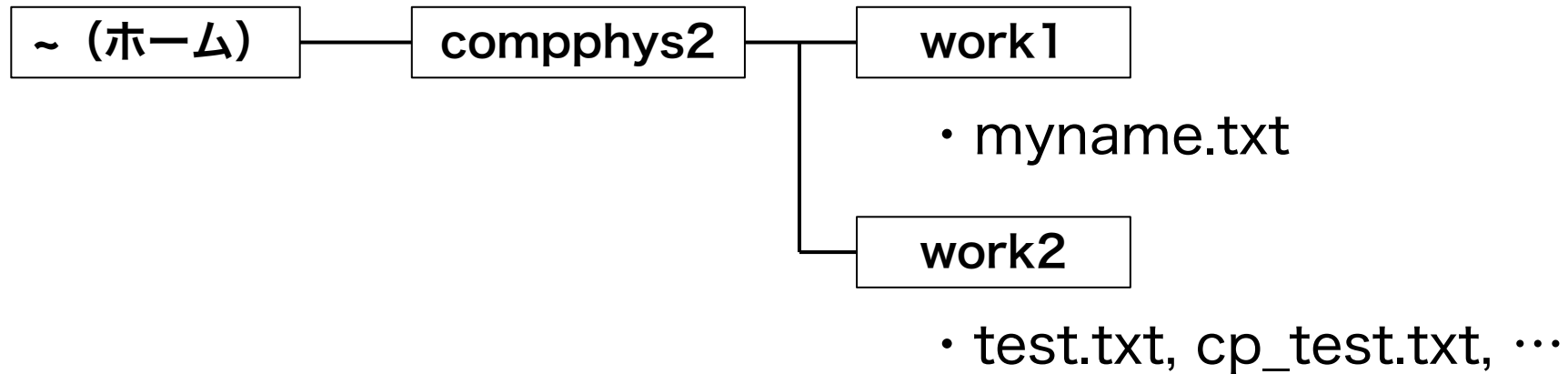
- ・先ほど実行したコマンドの構文について
 - ・“./”はカレントディレクトリ（今自分がいるディレクトリ）を意味する
 - ・“./test.txt”は「カレントディレクトリの中のtest.txt」を意味する
- ・カレントディレクトリ内での操作では“./”を省略することもできます
 - ・試しに以下のコマンドを実行してみましょう

```
akiyama@~/compphys2/work2$ cp test.txt cp2_test.txt
```

- ・lsを使って“cp2_test.txt”が作成されたことを確認してみましょう
- ・catを使って“cp2_test.txt”の中身も確認しましょう

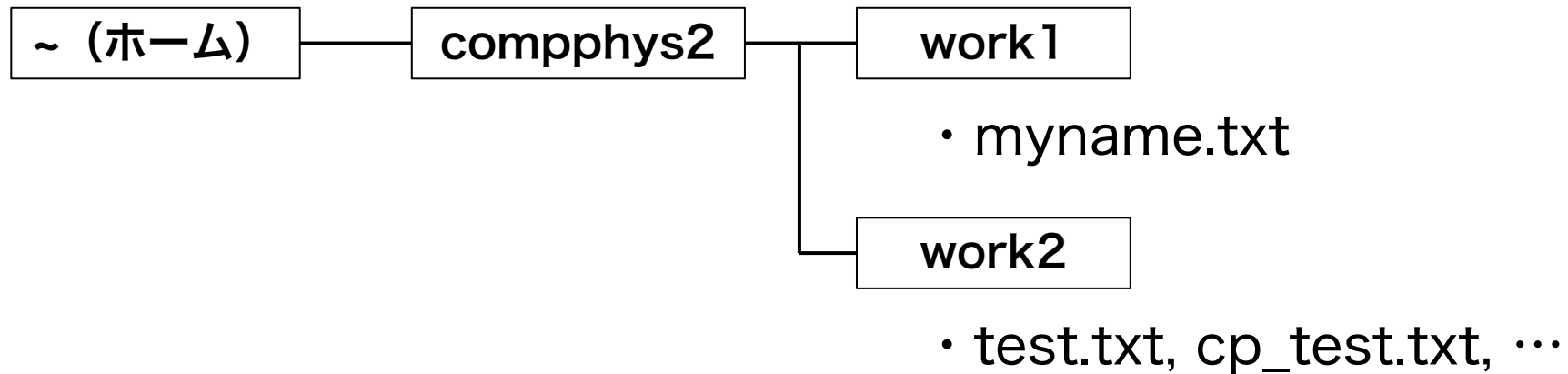
ディレクトリを跨ぐファイルのコピー

- ・ 今度は, 前回作成した“myname.txt”を今いるディレクトリにコピーしてみましょう
- ・ まず, 現状のディレクトリ構造は以下のようにになっています



方法1：絶対パスを使う

- ・先ほど出てきた「パス」には絶対パスと相対パスの二種類があります
- ・絶対パスとは、ディレクトリのツリー構造のトップを起点とした記述形式



- ・絶対パスによるmyname.txtの所在の記述
 - ・“~/compphys2/work1/myname.txt”
 - ・「ホームの中のcompphys2の中のwork1の中のmyname.txt」という意味

方法1：絶対パスを使う

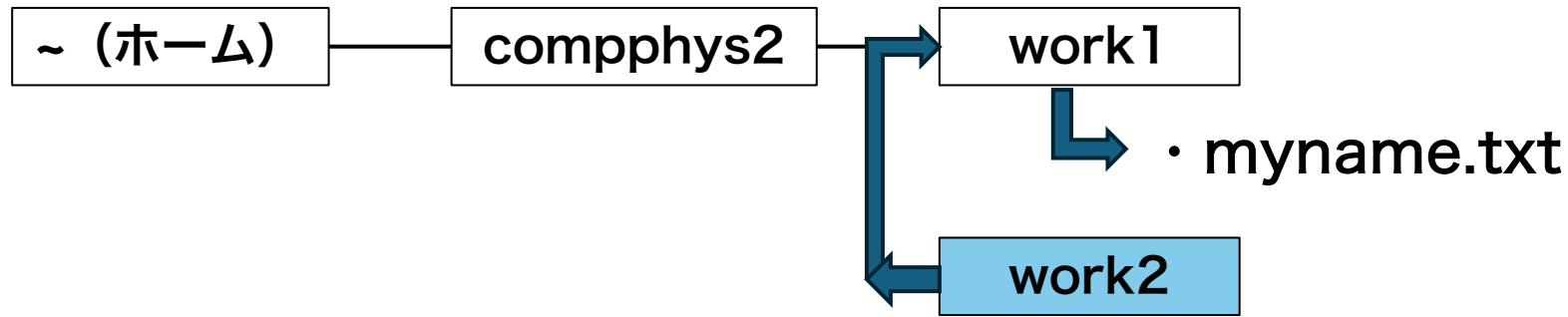
- ・ “cp コピーしたいファイルのパス コピー先のファイルのパス” でコピーできます
 - ・ ここでは、コピー後のファイル名はmyname.txtのままとします
 - ・ コピー後のファイル名を変えない場合、コピー先のファイルのパスの部分は、単にカレントディレクトリ（./）でOK

```
akiyama@~/compphys2/work2$ cp ~/compphys2/work1/myname.txt ./
```

- ・ lsを使い、“myname.txt”がwork2の中にコピーされたことを確認しましょう
- ・ catを使い、コピーされた“myname.txt”の中身も確認しましょう

方法2：相対パスを使う

- ・ 相対パスとは, カレントディレクトリを起点とした記述形式



- ・ test.txt, cp_test.txt, ...
- ・ work2から見た相対パスによるmyname.txtの所在の記述: “../work1/myname.txt”
 - ・ “../”はカレントディレクトリから見て一つ上のディレクトリ（compphys2）
 - ・ “../work1/myname.txt”で「一つ上のディレクトリの中のwork1の中のmyname.txt」という意味になります

方法2：相対パスを使う

- ・ “cp コピーしたいファイルのパス コピー先のファイルのパス”でコピーする
 - ・ コピー後のファイル名はcp_myname.txtとしましょう

```
akiyama@~/compphys2/work2$ cp ../work1/myname.txt cp_myname.txt
```

- ・ lsを使い, “cp_myname.txt”がwork2の中に作成されたことを確認しましょう
- ・ catを使い, コピーされた“cp_myname.txt”の中身も確認しましょう

cpコマンドの注意点

- ・ 上書きに注意

- ・ 先ほどコピーしてきたcp_myname.txtを編集し, 入力されている文字を全て削除し, 保存しましょう
- ・ 保存したらターミナルに戻り, catを使ってcp_myname.txtを見てみましょう
 - ・ 何も書かれていないため, 以下のようになります

```
akiyama@~/compphys2/work2$ cat cp_myname.txt
```

cpコマンドの注意点

- ・ 上書きに注意

- ・ この状態で, もう一度以下のコマンドを実行し, myname.txtをすでにコピーしてきた cp_myname.txtと同じ名前でコピーしてみます
- ・ 再度コマンドを打ち直してもいいですが, 上矢印キーを押すと過去に入力したコマンドに戻ることができます (同じコマンドを打つ手間が省けます)
- ・ 上矢印キーを何度か押して, コマンドを復元しましょう (行き過ぎたら下矢印で戻れます)

```
akiyama@~/compphys2/work2$ cp ../work1/myname.txt cp_myname.txt
```

- ・ catを使ってcp_myname.txtを見てみましょう

```
akiyama@~/compphys2/work2$ cp ../work1/myname.txt cp_myname.txt
akiyama@~/compphys2/work2$ cat cp_myname.txt
Shinichiro Akiyama
```

- ・ このように, 同じファイル名でcpすると自動的に上書きされるので気をつけましょう

絶対パスと相対パスの使い分け

- ・ 絶対パスはカレントディレクトリに依存しない記述形式
 - ・ いわば各ディレクトリや各ファイルの住所
 - ・ リンクを貼る時には便利
- ・ 相対パスはカレントディレクトリに依存した記述形式
 - ・ 実際の作業を進める時には相対パスをよく使う

カレントディレクトリの絶対パスの取得：pwdを使う

- ・カレントディレクトリの絶対パスを知りたい場合はpwdコマンドを使う

```
akiyama@~/compphys2/work2$ pwd  
/Users/akiyama/compphys2/work2
```

- ・ここではホームディレクトリは“~/”ではなく“/Users/”で表示されています
(皆さんの環境では“/home/”となっていると思います)

ディレクトリのコピー：cpコマンドを使う

- ・ 今度は, 前回作成した“work1”（ディレクトリ）を今いるディレクトリにコピーしよう
 - ・ ディレクトリをコピーする方法
 - ・ “cp **-r** コピーしたいディレクトリのパス コピー先のディレクトリのパス”
 - ・ ディレクトリの構造（p.13）を思い出し, 相対パスでコピーしてみます

```
akiyama@~/compphys2/work2$ cp -r ../work1 ./previous_work1
```

- ・ ここでは, “work1”を“previous_work1”という名前でコピーしました
- ・ lsを使って“previous_work1”というディレクトリが出来ているか確認しましょう
- ・ “previous_work1”の中に入って, “myname.txt”が入っているかも確認しましょう

```
akiyama@~/compphys2/work2$ ls
cp2_test.txt    cp_myname.txt  cp_test.txt    myname.txt     previous_work1 test.txt
```

ディレクトリのコピー：cpコマンドを使う

- “cp **-r** コピーしたいディレクトリのパス **コピー先のディレクトリのパス**”
 - cp後ろにつけた“-r”はcpコマンドに対するオプションと呼ばれます
- Linuxの場合、以下のようにして各コマンドに対して用意されているオプションを調べられます

helpの前は-(ハイフン)が二つ！

- “コマンド **--help**”
- 例えば、ターミナルで“cp --help”を実行してみましょう
- オプションのリストが見えればOKです

その他のよく使うオプション一例

- lsコマンド

- 一度ホームディレクトリに戻りましょう（その場でcdする）
- ホームディレクトリでlsを実行

```
akiyama@~$ ls
Applications  Downloads  Movies      Public      dev          mypy
Desktop       Dropbox    Music       Zotero      dev_claude  "
Documents     Library    Pictures    compphys2   intel
```

- 今度はlsに-laオプションをつけて実行してみましょう
 - 表示形式が変わり, lsでは見えていなかったディレクトリやファイルが見えたらOK
 - -laは, -l（詳細情報表示）と-a（隠しファイルを含む全てを表示）というオプションを合わせたもので, 使用頻度の高いコマンドです
 - 隠しファイルとは名前が“.”から始まるもの

ファイルの移動：mvコマンドを使う

- ・まずホームから“work2”のディレクトリまで移動しましょう
- ・以下の手順で, “cp2_test.txt”を“work2”の中にコピーしてきた“previous_work1”へ移動させてみましょう
- ・ファイルを移動するにはmvコマンドを使う
 - ・コマンドの構造は, “mv **移動したいファイルのパス** **移動先のファイルのパス**”
- ・以下のコマンドを実行してみよう

```
akiyama@~/compphys2/work2$ mv ./cp2_test.txt ./previous_work1
```
- ・lsを使って, “work2”の中に“cp2_test.txt”が無いことを確認しましょう
- ・今度は“previous_work1”に移動してlsし, “cp2_test.txt”が有ることを確認しましょう
- ・cpとmvの違いを理解しましょう

ファイル名の変更：mvコマンドを使う

- ・ mvコマンドを応用するとファイル名の変更が可能
- ・ “previous_work1”に移動させた“cp2_test.txt”を“test.txt”という名前に変更してみましょう
- ・ ファイル名の変更とはファイルの移動の一種です

```
akiyama@~/compphys2/work2/previous_work1$ mv cp2_test.txt test.txt
```

- ・ lsを使って, “cp2_test.txt”が“test.txt”という名前に変更されたことを確認しよう

ディレクトリの移動：mvコマンドを使う

- ・ 今度は, “work2”の中にある“previous_work1”を“compphys2”の真下へ移動させてみましょう
- ・ ここでは, まずは“compphys2”まで戻っておくことにしましょう

```
akiyama@~/compphys2/work2/previous_work1$ cd ../../
```

- ・ 相対パスを使って, 次のように移動させましょう

```
akiyama@~/compphys2$ mv ./work2/previous_work1 ./
```

- ・ lsを使って, “previous_work1”が“compphys2”直下に移動されたのを確認しましょう
 - ・ cpでディレクトリのパスを扱うには-rオプションが必要だったが, mvでは不要です
- ・ 移動させた“previous_work1”の中に入り, lsで中身も確認しておきましょう

ファイルやディレクトリの削除：rmコマンドを使う

- ・ ファイルやディレクトリの削除は, rmコマンドを使う
 - ・ rmで削除すると元には戻せないなので, 実行時は要注意！
- ・ “previous_work1”直下にある“test.txt”を削除してみます
 - ・ -iオプション：削除前に本当に削除して良いか確認する
 - ・ 次のコマンドを実行

```
akiyama@~/compphys2/previous_work1$ rm -i test.txt  
remove test.txt? y
```
 - ・ Terminalが削除していいか確認してくる
 - ・ 削除する場合: yを押してEnterを押す（今回はこちら）
 - ・ 削除しない場合: nを押してEnterを押す
- ・ lsで“test.txt”が消えたことを確認したら, “compphys2”の真下へ移動しておきましょう

ファイルやディレクトリの削除：rmコマンドを使う

- ・ディレクトリの削除は, rmコマンドに-rオプションをつける
 - ・-iオプションと組み合わせる場合は-riとする (-ir, -r -iでもOK)
- ・“compphys2”直下にある“previous_work1”を削除してみます
- ・次のコマンドを実行 `akiyama@~/compphys2$ rm -ri previous_work1`
- ・Terminalが削除していいか確認してくる
 - ・ディレクトリ配下に入るか尋ねてきた場合もyを押してEnter
 - ・その場合, 配下のファイルを削除するか逐一尋ねられるので, 逐一yを押してEnter
 - ・配下のファイルが全て削除されたら, 最後にディレクトリを削除するか尋ねてくるので, yを押してEnter
- ・lsで“previous_work1”が消えたことを確認しましょう

本日の演習内容

- コマンドを使ったファイル操作について
 - 前回の復習といくつかの新しいコマンド (cp, mv, rm…)
- matplotlibによる図の作成
 - 関数のプロット
 - 数値データのプロット
 - スクリプトの活用

matplotlibとは？

- Pythonでグラフを描くためのパッケージ
 - 特によく使うのが, matplotlib.pyplotモジュール
 - MATLAB-likeな環境でグラフ描画ができる
 - MATLAB（マトラボ）は数値解析向けのソフトウェア/プログラミング言語の一つ
 - 公式のUser manualは[ここ](#)から
 - Pythonによるプログラミングについては後半の演習で詳しく学びます
 - 簡単な資料を~/compphys2/pythonの中に置いています：
- ```
akiyama@~/compphys2/python/quick_ref_python$ xdg-open quick_ref_python.pdf
```
- 今回の演習では, サンプルスクリプトをベースに, matplotlibの使用感を体験しましょう

# Pythonスクリプトファイル（詳しくは後日の演習で）

- Pythonインタプリタには二種類ある
  - ① 対話モード
  - ② Pythonスクリプトファイルを使う方法
- ②の方法では、スクリプトファイルのパスを引数にして、python3コマンドを実行する
  - Pythonスクリプトファイルの拡張子は.py
- 以降では②の方法を使っていきます

# データを読み込んでmatplotlibでプロットしてみよう

- まず“lecture2”の中に移動し, lsをして“src”と“data”というディレクトリがあることを確認しましょう
- “src”の中に演習で使うPythonのソースコード, “data”の中に演習で使うデータが格納されています
- lsで中身を見ておきましょう

```
akiyama@~/compphys2/lecture2$ cd src
akiyama@~/compphys2/lecture2/src$ ls
sample_plot_data.py
akiyama@~/compphys2/lecture2/src$ cd ../data
akiyama@~/compphys2/lecture2/data$ ls
population.dat
```

- “sample\_plot\_data.py”がソースコード, “population.dat”がデータファイルです

# データを読み込んでmatplotlibでプロットしてみよう

- ・ 続いて, ソースコードとデータを演習用ディレクトリ“work2”へcpしてきましょう
  - ・ “work2”へ移動し, 相対パスを使って次のようにcpしていきましょう

```
akiyama@~/compphys2/work2$ cp ../lecture2/src/sample_plot_data.py ./
akiyama@~/compphys2/work2$ cp ../lecture2/data/*.dat ./
```

- ・ 二つ目のcpで, 「\*.dat」の\*はワイルドカードと呼ばれます
- ・ \* (ワイルドカード) は0文字以上の任意の文字列を意味します
- ・ これにより, ファイル名が.datで終わる全てのファイルが一撃でcpされます
- ・ lsを使って, ファイルがcpされていることを確認しましょう

# サンプルスクリプトを眺めておきましょう

- ・ cpしたsample\_plot\_data.pyをVS codeで開いてみましょう

```
akiyama@~/compphys2/work2$ code ./sample_plot_data.py
```

- ・ 以下のスライドでサンプルスクリプトに関する簡単な説明をしておきます
  - ・ 繰り返しになりますが, Pythonは後日学んでいくので, 今日の時点では気軽にソースコードを使いましょう

# サンプルスクリプトの説明1

- ・最初の二行はこのスクリプトで使うパッケージを読み込んでいます
  - ・ as以下で読み込んだパッケージに略称をつけています
- ・以下のコードで plt.~~~ という部分が出てきたら, それはmatplotlib.pyplotで提供される描画に関連した機能を使用していることを意味します

```
1 import matplotlib.pyplot as plt
2 import numpy as np
```

- ・ もう一つnumpyというパッケージも導入しています
  - ・ numpyについては後半の授業で詳しく説明します

## サンプルスクリプトの説明2

- ・ 4行目以降のコードブロックが、データファイルを読み込んで図を返す機能（関数）になっています（この部分は少し口頭で補足説明をするかもしれません）

```
4 def plot_dat(input_filename, output_filename):
5 data = np.loadtxt(input_filename)
6 x, y = data[:, 0], data[:, 1]
7 plt.figure(figsize=(8, 6))
8 plt.title(f"Visualization of {input_filename}", fontsize=20)
9 plt.xlabel("x", fontsize=20)
10 plt.ylabel("y", fontsize=20)
11 plt.plot(x, y, "o-")
12 #plt.errorbar(x, y, yerr=yerr, fmt="o", color = "b", capsize=3)
13 plt.grid(True)
14 plt.savefig(output_filename)
15 plt.close()
```

## サンプルスクリプトの説明3

- ・最後に, 17行目の一文がこのサンプルスクリプトの主要部分です
  - ・先ほどの関数を呼び出し, input.datを読み込んでoutput.pdfとして図を返します

```
17 plot_dat("input.dat", "output.pdf")
```

# 演習で使う数値データの説明

- ・ 続いてデータファイルの“population.dat”について説明します
- ・ “population.dat”には、大正9年から平成27年までの全国、及び各都道府県の人口が格納されています（5年ごと）
- ・ “population.dat”は、[ここ](#)からダウンロードできる国勢調査のオープンデータに基づいています
- ・ catを使い、“population.dat”を見てみましょう

The screenshot shows the e-Stat website interface. At the top, there's a header with the e-Stat logo and navigation links. Below the header, there's a search bar with filters for 'File', 'Census', 'Time Series Data', and 'CSV format'. The search results show a table with columns for 'Dataset Name', 'Dataset Code', and 'Survey Overview'. The first row is for 'Census' (国勢調査) with code '00200521'. The overview text states that the census is conducted every 5 years and provides population data for all prefectures and municipalities in Japan.

| 政府統計名   | 国勢調査                                                                                                                                                                                                                   | 詳細 |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 政府統計コード | 00200521                                                                                                                                                                                                               |    |
| 調査の概要   | <p>国勢調査は、日本に住んでいるすべての人と世帯を対象とする国の最も重要な統計調査で、5年ごとに実施されます。国勢調査から得られる日本の人口や世帯の実態は、国や地方公共団体の行政において利用されることはもとより、民間企業や研究機関でも広く利用され、そのような利用を通じて国民生活に役立てられています。</p> <p>国勢調査では、年齢別の人口、家族構成、働いている人や日本に住んでいる外国人などの結果を提供しています。</p> |    |

# 演習で使う数値データの説明

- “population.dat”の構造は次のようになっています

- 1列目：都道府県コード
- 2列目：都道府県名
- 3列目：元号
- 4列目：和暦（年）
- 5列目：西暦（年）
- 6列目：人口（総数）
- 7列目：人口（男性）
- 8列目：人口（女性）

```
akiyama@~/compphys2/work2$ cat population.dat
0 Japan Taisho 9 1920 55963053 28044185 27918868
1 Hokkaido Taisho 9 1920 2359183 1244322 1114861
2 Aomori Taisho 9 1920 756454 381293 375161
3 Iwate Taisho 9 1920 845540 421069 424471
4 Miyagi Taisho 9 1920 961768 485309 476459
5 Akita Taisho 9 1920 898537 453682 444855
6 Yamagata Taisho 9 1920 968925 478328 490597
7 Fukushima Taisho 9 1920 1362750 673525 689225
8 Ibaraki Taisho 9 1920 1350400 662128 688272
9 Tochigi Taisho 9 1920 1046479 514255 532224
10 Gunma Taisho 9 1920 1052610 514106 538504
11 Saitama Taisho 9 1920 1319533 641161 678372
12 Chiba Taisho 9 1920 1336155 656968 679187
13 Tokyo Taisho 9 1920 3699428 1952989 1746439
14 Kanagawa Taisho 9 1920 1323390 689751 633639
15 Niigata Taisho 9 1920 1776474 871532 904942
16 Toyama Taisho 9 1920 724276 354775 369501
17 Ishikawa Taisho 9 1920 747360 364375 382985
18 Fukui Taisho 9 1920 599155 293181 305974
19 Yamanashi Taisho 9 1920 583453 290817 292636
20 Nagano Taisho 9 1920 1562722 758639 804083
21 Gifu Taisho 9 1920 1070407 536334 534073
22 Shizuoka Taisho 9 1920 1550387 774169 776218
23 Aichi Taisho 9 1920 2089762 1033860 1055902
24 Mie Taisho 9 1920 1069270 525957 543313
25 Shiga Taisho 9 1920 651050 313737 337313
26 Kyoto Taisho 9 1920 1287147 650780 636367
```

## 特定の数値データを抜き出す

- “population.dat”から特定の情報を抜き出し、可視化してみましょう
  - 例題：「横軸に西暦, 縦軸に日本の総人口を取って, 総人口の推移を見よう」
- ここで, Terminalから使える便利なコマンドを紹介します
  - **grep**コマンド=あるファイルから特定の文字列を含んだ行だけを抜き出す
  - “grep”**“抜き出したい文字列”** **ファイル (のパス)** ”
  - 文字列はダブルクォーテーション (“...”）あるいはクォーテーション (‘...’) で囲む必要があります
- 以下のコマンドを実行してみましょう

```
akiyama@~/compphys2/work2$ grep "Japan" population.dat
```

- “population.dat”のうち, “Japan”の文字列を含む行だけが出力されます

## 特定の数値データを抜き出し, 保存する

- ・ さらに, 次のようにgrepコマンドを使うと, 抜き出した結果を新たに保存できます
  - ・ “grep **“抜き出したい文字列”** **ファイル (のパス)** **>>** **保存ファイル (のパス)** ”
- ・ 以下のコマンドを実行してみましょう (↑キーを押すと便利)

```
akiyama@~/compphys2/work2$ grep "Japan" population.dat >> japan.dat
```

- ・ catを使い, “japan.dat”を見てみよう

# サンプルスクリプトを使うために、さらにデータを整形

- ・配布しているサンプルスクリプトでは、データファイルの1列目をx軸の値, 2列目をy軸の値として読み込みます

```
data = np.loadtxt(input_filename)
x, y = data[:, 0], data[:, 1]
```

- ・ Pythonは0始まりなので, 1列目のことを0列目, 2列目のことを1列目と呼んでいます
- ・ 今の場合, Terminalから使える便利なコマンドをもう一つ紹介します
  - ・ **awk**コマンド=テキストファイルを列ごとに処理・操作する

ここはクォーテーション

- ・ “awk **{print \$1, \$2}** **ファイル (のパス)** **>> 保存ファイル (のパス)** ”
  - ・ 上記のコマンドにより, 「ファイル」の1列目と2列目だけが抽出され, 「保存ファイル」の名前で保存される
  - ・ このコマンドでの列数カウントは0始まりではなく, 1始まりです

# サンプルスクリプトを使うために, さらにデータを整形

- ・「横軸に西暦, 縦軸に日本の総人口を取って, 総人口の推移を見よう」という課題なので…
  - ・先ほど行抽出をした“japan.dat”の5列目と6列目が抽出したい
- ・以下のようにawkコマンドを実行し, 列抽出の結果を“japan\_year\_vs\_total.dat”という名前で保存しましょう

```
akiyama@~/compphys2/work2$ awk '{print $5, $6}' japan.dat >> japan_year_vs_total.dat
```

- ・catを使って所望の列だけが抽出されたことを確認しておきましょう

# サンプルスクリプトを使って作図します

- ・ サンプルスクリプトのメイン部分である17行目だけを以下のように編集します

```
17 plot_dat("japan_year_vs_total.dat", "japan_year_vs_total.pdf")
```

- ・ 読み込ませたい“japan\_year\_vs\_total.dat”を第一引数に, 作図結果をpdfとして保存する際のファイル名を第二変数に記述します
  - ・ 上の例では読み込ませるファイル名にあわせて, 図も“japan\_year\_vs\_total.pdf”としてみましたが, 図を保存する際のファイル名は完全に任意です
- ・ 17行目の編集が終わったら一度サンプルスクリプトを保存しましょう
  - ・ 保存をしないと編集内容が反映されないので要注意です

# サンプルスクリプトを使って作図します

- ・ いよいよサンプルスクリプトを実行します
  - ・ 以下のように, スクリプトファイルを引数として, python3コマンドを実行します

```
akiyama@~/compphys2/work2$ python3 sample_plot_data.py
```

- ・ コマンドが通ったら, lsでカレントディレクトリの中をチェックしましょう

```
akiyama@~/compphys2/work2$ ls
cp_mynome.txt japan_year_vs_total.dat noisy_data1.dat population.dat
cp_test.txt japan_year_vs_total.pdf noisy_data2.dat sample_plot_data.py
japan.dat myname.txt noisy_data3.dat test.txt
```

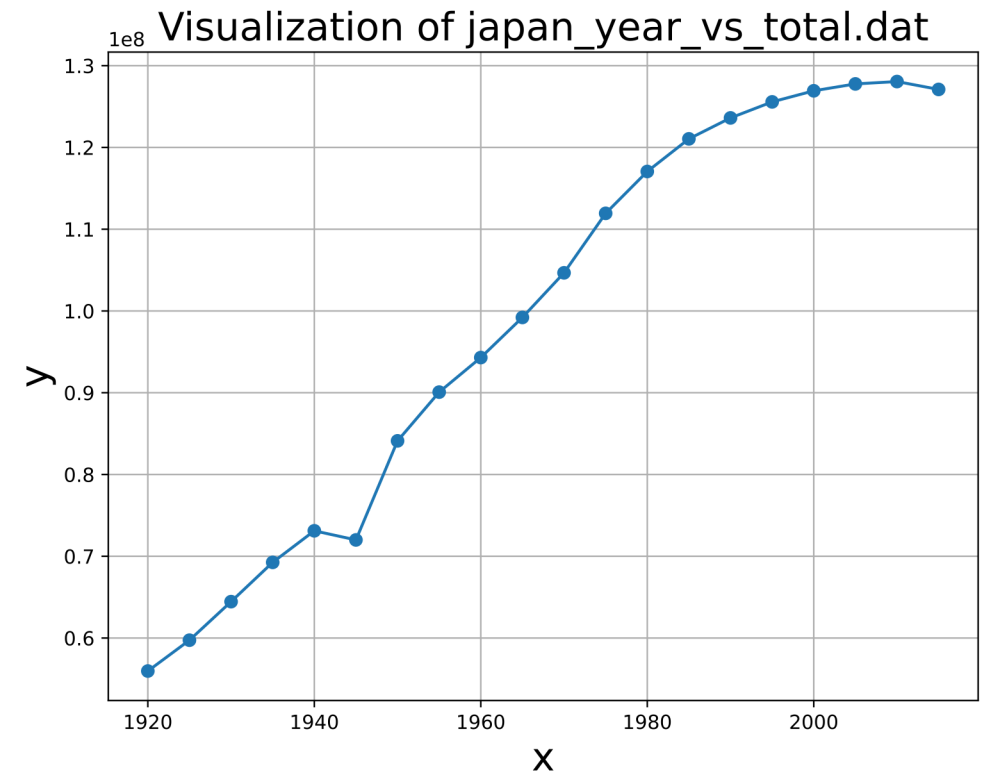
- ・ サンプルスクリプトで指定した名前のファイルが出来ていれば成功です

# サンプルスクリプトを使って作図します

- ・ 出来上がったpdfファイルを開いてみましょう

```
akiyama@~/compphys2/work2$ xdg-open japan_year_vs_total.pdf
```

- ・ 右のような図が見えましたか？



# ここまで終わったら

- お疲れ様でした
- [宿題1] 自分の出身都道府県の総人口の推移も可視化してみましょう
- [宿題2] “population.dat”の7列目と8列目はそれぞれ男性, 女性の人口です. awkコマンドを使って, それぞれの列を抽出し, 日本の男性人口の推移と女性人口の推移も可視化してみましょう